

Where Coding Agents Actually Fail:

A Measurement Blind Spot, a Bounded Localisation Effect, and a Working Router

Ziheng Yu Xuhao Chen

September 13, 2026

Abstract

Coding agents fail, and a large literature tries to detect *when*. This study asks a prior question: **are we measuring the right thing?** From 236,137 mechanically labelled edit operations across 25,681 agent runs on 1,213 software-engineering instances we report four results, one of which corrects our own earlier headline.

(1) A measurement blind spot. A commonly computable statistic — the share of a run’s edits aimed at files in *that run’s own final patch* — inverts the sign of the solved-versus-failed comparison. It reports failed runs localising *better* (0.673 vs 0.585); against an **independent gold patch** the direction reverses (0.477 vs 0.407). The metric is a closed loop, so a run that fixates on the wrong file and never wavers scores 1.0. Two published papers report the direction our correction recovers, and we tested three mechanisms for the inversion — *all three failed*, so the defect is demonstrated but not explained.

(2) Localisation is bounded. The robust measure is binary — did the run ever edit a gold-patch file — and it is positive in every patch-width stratum: 98.2% of successful runs and 69.1% of failed runs reach the correct file. Localisation can therefore address at most the 30.9% of failures that never reach it.

(3) The causal test. Holding a defect fixed and varying *only* how easy it is to find: handing over the exact file and function lifts success from 23.5% to 43.8%, a failure-rate drop of 0.203 that is *below* the 0.309 ceiling the observational result predicted in advance. Decisively, **even when the agent reached the correct file — 82% of runs did — it succeeded only 28.6% of the time.** A re-diagnosis prompt did not help.

(4) A working router. From the first 10–60% of a run we predict whether a struggling agent needs *search* or *verification*, reaching 0.676–0.751 at separating lost from wrong-fix runs, where every published-style heuristic (loop, burst, redundancy, output-shape) sits **at or below chance** (0.407–0.539). Its false-alarm rate is genuinely controlled ($\alpha = 0.05 \rightarrow 0.046$ achieved), unlike a circular calibration claim we withdrew; as a routing policy it is worth 0.764–0.800 against 0.732 for always-verify and 0.518 for always-search.

We report a ledger of **eight retracted claims, three of them our own headlines**, and we state plainly that the qualitative interpretation — that capable agents fail *after* reaching the right code — is prior art, independently reproduced here rather than discovered.

Keywords: coding agents, failure analysis, measurement validity, runtime monitoring, reproducibility, honest reporting.

1 Introduction

Coding agents resolve a substantial share of real software issues and fail on the rest. Much work treats failure as a *monitoring* problem: watch the trajectory, notice when the agent is stuck, intervene. That framing has a hidden premise — that the agent fails because it is **lost**, unable to find the right place to work.

This study began with a startling result supporting the opposite view. On tasks containing both a solved and a failed run, failed runs appeared to aim a *higher* share of their edits at the file in their final patch (0.620) than solved runs did (0.576), $p = 3.3 \times 10^{-4}$. If failures localise at least as well as successes, localisation is not the bottleneck and a great deal of monitoring research is aimed at the wrong target.

That result is wrong, and §3 shows why. But the way it is wrong, and what stands in its place, are more useful than the original claim.

We had two advantages. First, the framework we study prints a structural footer into its observations — `[File: /abs/path/to/file.py (N lines total)]` — in 95.8% of edit steps. That lets us label every edit *mechanically*, with no human judgement and no model in the loop: whether the file changed, by how many lines, and whether those lines ever reach the agent’s final patch. The study moves from “*we asked a model what stagnation looks like*” to “*we counted which written lines shipped.*”

Second, we obtained **independent ground truth**: gold patches for 927 of 1,213 instances, from the upstream SWE-rebench and SWE-bench corpora. That target does not depend on anything an agent produced — and it is what exposed the blind spot.

Contributions. (i) a measurement blind spot, demonstrated with both metrics on identical edits and a control showing they agree where they must (§3); (ii) a bounded localisation result, robust across every stratum, replacing a retracted claim (§4); (iii) a causal test with localisation solved by fiat (§5); (iv) a router that beats every baseline on a target where the field’s detectors have no signal, with genuine false-alarm control (§6); (v) a retraction ledger (§8).

2 Related work, and what is not ours

We verified every citation below by fetching it directly; automated reconnaissance summaries were not trusted for load-bearing claims.

The interpretation is not ours. *Coherence Collapse* [1] reports across 16,758 trajectories that “*the dominant failure of capable models is not localization: 60–69% of failures on SWE-Agent and OpenHands reach and edit the correct functions yet still produce incorrect patches.*” Our independently measured 67.1% **sits inside that range**. A second study [2] finds the majority of failing trajectories locate the correct files. We claim **no novelty** for the qualitative finding, and say so rather than re-deriving it.

Two papers report the opposite direction on adjacent metrics. FailForge [3] reports localisation precision higher for passing than failing runs, and TraceProbe [4] finds same-task file-selection divergence in failed runs. These agree with **our corrected** result and disagree with the metric we expose in §3. Reconciling an apparent contradiction in the literature is part of the contribution.

The monitoring framing is not ours either. *Confident and Wrong* [5] shows GPT-5 submitting a patch on 100% of runs while resolving 44%, and observes that “*completion-based and consistency-based monitoring both look healthy exactly when the agent should not be trusted.*” Our finding that the *most* fixated runs are precisely those that never reached the right file (§3) independently corroborates that warning.

Waste has been measured before. TRIM [6] reports 20.0% “CodeSlop” on SWE-bench, close to our 19.1% dead-end rate. The objects differ and we state the difference: TRIM measures unnecessary *lines in a passing patch*, established by counterfactual re-execution; we measure edit *operations* that never reach the patch, with no execution. Ours is cheaper and coarser; theirs is causally stronger.

The detection bar is soft. Published detectors sit at AUC 0.6–0.7 [7]; the field’s shared attribution benchmark reaches 53.5% agent-level but only 14.2% step-level [8]; a failure-as-a-process monitor achieves 82% precision at 2–3% false alarms but only 18.2% recall [9]; the step-level redundancy ceiling is 24.88% [10].

What is ours is the measurement result: no located paper reports failed runs localising *better* within-instance, and a broken self-referential metric is exactly what produces that sign.

3 The measurement blind spot

3.1 The statistic

The conventional localisation statistic is

$$\text{on-target} = \frac{\text{edits aimed at files the run’s final patch touches}}{\text{edits}},$$

Both the numerator’s file set and the denominator come from **the same run**. It asks “*did the run converge on what it produced?*”, not “*did it aim at the right place?*” A run that fixates on the wrong file and never wavers scores 1.0.

3.2 Replacing the target with ground truth

We obtain gold patches for 927 of 1,213 instances (843 SWE-rebench, 84 SWE-bench) and recompute on **identical edit steps**.

measure	solved	failed	reading
on-target, own patch (<i>the conventional statistic</i>)	0.594	0.670	failed “better”: the startling result reproduces
on-target, gold patch (basename)	0.477	0.407	direction reverses
on-target, gold patch (path match)	0.510	0.430	robust to the match rule
ever reached a gold file	0.982	0.670	large and unambiguous

Table 1: The same edit steps under two target definitions, *pooled over runs* (`wrongness.json`). The conventional statistic and the independent one disagree in *sign*. Within-instance paired means for the gold target are 0.496 solved vs 0.448 failed ($p = 0.007$, 228 instances).

3.3 A control that both metrics are sane

Restrict to runs whose own patch is a *single* file that *is* a gold file. There the two target sets must coincide, and they do: 0.615 vs 0.638, mean absolute gap 0.033 ($n = 8,801$). Neither definition is broken; they measure different things.

3.4 The robust measure, and why the shares are not

Both *shares* are fragile to patch composition; the *binary* measure is not.

own patch width	n solved	n failed	ever reached gold (solved)	(failed)	Δ
1 file	2,555	9,070	0.992	0.691	+0.301
2 files	593	3,688	0.963	0.734	+0.229
3 files	111	1,010	0.973	0.697	+0.276
4+ files	49	809	0.694	0.621	+0.073

Table 2: The binary measure is positive in *every* stratum. The share-based gold measure is +0.070 pooled but only +0.020 after standardising over patch width — 72% of the gap is composition — and flips sign inside strata 2–4. We therefore quote the binary measure and refuse to quote the share.

3.5 Three mechanisms tested; all three falsified

We did not accept “the metric is broken” without trying to say *how*.

1. **Confident wrongness** — a run fixating on a *wrong* file scores high. **Falsified:** runs that never reached a gold file score *lower* (0.661) than those that did (0.679), the 95% CI excluding zero on the wrong side.
2. **Patch breadth** — failed runs write broader patches (2.20 files vs 1.35), enlarging their own target set. **Falsified as the explanation:** standardising over patch width makes the gap *larger* ($-0.088 \rightarrow -0.125$), and the gap is already largest in the narrowest stratum, where breadth cannot operate.
3. **Fixation** — the metric rewards concentrating edits on one file. **Partly supported, not the mechanism:** failed runs are more fixated (0.779 vs 0.727) and the most fixated runs of all are those that never reached the right file (0.818 vs 0.759) — independently corroborating [5] — but fixation correlates no more with the broken metric (0.251) than with the gold metric (0.294).

The honest conclusion is a **construct-validity** one, not a mechanism one. We demonstrate the inversion, prove the metrics agree where they must, and show the correction restores agreement with the literature. We cannot explain it. The mechanism remains **open**.

3.6 Two independent replications on eight unseen shards

The frozen study used Nebius shards 0–3. Shards 4–7 and 8–11 were separately downloaded and parsed into their own tables, so no replication touches the frozen artifacts, with gold patches resolved independently (74.6% and 77.9% coverage). Together the three tables cover **12 of 12 shards and 80,035 runs**:

set	shards	runs	self solved → failed	gold solved	failed	p	wrong-fix
frozen	0–3	26,679	0.594 → 0.670	0.477	0.407	7.0×10^{-3}	67.0%
held-out A	4–7	26,680	0.593 → 0.677	0.492	0.421	3.6×10^{-3}	69.0%
held-out B	8–11	26,676	0.614 → 0.686	0.512	0.415	2.8×10^{-5}	66.4%

Table 3: Three disjoint sets of ~26,000 runs each. The broken metric inverts in every one; the gold target reverses it back in every one, with the paired test *strengthening* on unseen data; and the wrong-fix share varies by only 2.6 points.

Scope, stated plainly. This is **not** a merged 80k-run table. It is full shard coverage delivered as one frozen table plus two held-out replications — methodologically stronger than a merge, since each replication is genuinely unseen, but it means the pipeline was never re-run on one 80k table. The earlier single-table attempt failed (two builds raced on one output file) and that failure is recorded rather than hidden. **No claim rests on a merged table.**

4 Where failures actually are

With a target independent of the agent, failure splits mechanically:

failure mode	definition	share
WRONG-FIX	failed, but <i>did</i> edit a gold file	67.1% (10,947 runs)
LOST	failed and never edited a gold file	30.9% (5,380 runs)

Stable across model scale (66.8% Llama-70B, 70.7% 8B, 70.2% 405B) and across gold patch widths (64.8–74.0%). Because only the binary measure is stratum-robust:

98.2% of successful runs and 69.1% of failed runs reach the correct file. Localisation tooling can address at most the 30.9% of failures that never get there; the remaining 69.1% happen with the agent already in the right place.

This is a *bounded-lever* claim and it is weaker than “localisation does not matter” — which is what our retracted result implied. The corrected version is also the one that agrees with the published literature instead of contradicting it.

5 The causal test

§4 is correlational. So we intervened, holding the defect fixed and changing *only* how easy it is to find.

Design. Real pure-Python packages, no Docker, the packages’ own test suites as verifier. Three arms, same tasks, same model, same budget: **unhinted** (a defect exists; find and fix it), **hinted** (told the exact file *and* function — localisation solved by fiat), and **verify** (unhinted plus “after a failed test, re-diagnose before changing more code”). Predictions were **fixed in advance** from §4 so a failure could not be reinterpreted afterwards.

Two harness faults had to be fixed first, both caught by reading transcripts, not scores. (i) 65.2% of turns were unparseable: our first protocol was ad-hoc text, but the model emits its own XML tool-call format regardless, so the pilot measured *protocol compliance*, not debugging. Native function-calling reduced this to 0–4%. (ii) **The agent was writing its own tests into tests/**, which pytest collects — an agent-authored passing test could have scored a run as *successful without the defect being fixed*. The verifier is now restored to the package’s original tests before final grading, as a real grader does.

Results. The larger run used a 48-task suite (8 real packages, 2 arms, 96 episodes, \$0.91). Auditing the raw episode records — not the summary — showed 13 episodes where the task was *not actually broken at the start*; those are excluded. On the remaining 83:

arm	n	success	95% CI	reached gold	success gold
hinted	40	0.550	[0.40, 0.70]	1.000	0.550
unhinted	43	0.372	[0.23, 0.51]	1.000	0.372

Table 4: Live experiment, temperature 0.

Every run in both arms reached the correct file — and 62.8% of the unhinted runs still failed. In this suite localisation was never the barrier at all, and the binding constraint is plainly the fix. This is the causal counterpart of the observational 69.1% / 98.2% gap, under a manipulation that removes search cost entirely.

P1 held. Handing over the exact file and function raised success by +17.8 points, and the *failure-rate* drop, 0.178, is below the 0.309 ceiling §4 predicted in advance.

The effect is not statistically significant, and we say so. Fisher’s exact test on 40 vs 43 episodes gives $p = 0.126$. P1 was pre-registered as an *inequality about magnitude* precisely so it could be checked without significance; a reader must not read it as a demonstrated improvement.

P3 held. 100% of runs reached the file and a majority still failed. An earlier, smaller run (16 tasks, 3 arms, 50 episodes) gave the same shape and carried the *verify* arm: hinted 0.438, verify 0.353, unhinted 0.235, with success-given-gold 0.583 / 0.400 / 0.286.

P2 failed and we report it as a failure. Telling the agent to re-diagnose after a failed test did *not* help; hinting was the strongest arm. The most obvious practical intervention this study could recommend does not work, and this is the second time a “spend the budget on verification” intuition failed under test in this project.

A behavioural finding of its own. In 71 of the 83 valid episodes the agent created or modified files under *tests/* — 1,048 files removed by the verifier restore in total. Agents write their own tests constantly while debugging. Because the verifier is restored before grading, an agent that “fixes” a test to make it pass is still scored as failing; but the frequency is itself a measurement, and it is exactly why our first harness would have produced a **wrong success measure**.

6 A router that works, on a target where the field has no signal

If 69.1% of failures happen in the right file, a runtime system facing a struggling agent should decide: spend on *search* or on *verification*? We predict which situation it is in, using **only the first 10–60% of a run**.

Integrity. The prefix is steps $0 \dots L - 1$; run length appears in no feature. A self-test that overwrites every post-cutoff step confirms all 112 feature columns are unchanged — it caught a genuine leak, now fixed. Folds are task-disjoint; every method is scored on identical rows and folds.

target	10%	20%	40%	60%
failure prediction	0.693	0.721	0.721	0.731
position baseline	0.667	0.664	0.666	0.666
output-shape (AgentStop-style)	0.554	0.565	0.594	0.607
lost vs wrong-fix	0.676	0.701	0.724	0.751
every baseline	0.407–0.493	0.411–0.490	0.413–0.486	0.413–0.539

Table 5: Baselines beaten in all 8 cells, bootstrap CIs excluding zero. On the lost-versus-wrong-fix target every published-style heuristic — loop, burst, redundancy, output-shape — is *at or below chance*.

A correction to our own prior claim. Our earlier finding that “step index beats every learned monitor” is invalid: at a fixed prefix fraction the position baseline is *exactly the run’s own length* (AUC matches run length to ≤ 0.006), a hindsight quantity no online system could use. Against a length-matched control the learned model holds at 0.63–0.76 and strictly dominates run length.

Calibration, done honestly. Our earlier “zero false alarms at every budget” claim was **circular** — label and detector were one statistic, so the numbers reported were literally an oracle’s. Here labels come from gold patches and detectors from prefix features, against an explicit null.

level	achieved false alarms	detection
$\alpha = 0.05$	0.046 (worst split 0.061)	0.160
$\alpha = 0.10$	0.078	0.258
$\alpha = 0.20$	0.123	0.386

8 of 8 levels controlled on the mean. A raw 0.5 threshold flags 98.4% of never-failing runs; the calibrated rule flags **3.39%**. A formal sequential e-value test is valid and **never fires** at $\alpha \leq 0.05$ — reported as a negative, not hidden.

The deployable result is the routing decision. Scoring 1 for a correct route:

policy	value ($\lambda = 0.25$)
this router	0.764–0.800
always verify	0.732
random	0.656
always search	0.518

It wins at every prefix fraction and every cost setting — 12 of 12 cells.

Limits we state rather than bury. Recall at a 5% alert budget is ≈ 0.058 for *every* method including the free baseline (base rate 0.837), so the usable output is a **ranking**, not a tight alarm. Gold matching is basename-only, 4.10% of edit steps carry no filename (biasing LOST upward), and the prefix cutoff uses hindsight, so the fully-online claim rests on the length-matched control.

7 Two instruments that did not deliver

7.1 The taxonomy does not transfer across scaffolds

We ported the mechanical labelling to four other corpora, first proving the port faithful by re-running it on SWE-agent and reproducing the frozen rates **exactly** (absolute difference 0.0 on all three rates, 236,137 edits, 25,681 runs).

Two conclusions, both negative and both load-bearing. First, **the instrument depends on a footer that exists in exactly one framework:** [File: ... (N lines total)] appears on 95.8% of SWE-agent edit

scaffold	editor footer	kept	revised	dead_end
SWE-agent (frozen reference)	0.958	0.157	0.652	0.191
OpenHands (SWE-Gym)	absent	0.734	0.229	0.037
PI agent	absent	0.821	0.060	0.118
mini-swe-agent-plus	absent	0.489	0.276	0.236
multi-framework mix	absent	0.706	0.115	0.179

Table 6: The taxonomy does *not* replicate. Dead-end rate spans a factor of 6 across scaffolds.

steps and **0%** of the others, so an edit’s *effect* is unmeasurable elsewhere and every mechanical number here is bounded to frameworks that print comparable state. Second, **the dead-end rate is scaffold-specific, not a property of coding agents** — it ranges from 3.7% to 23.6%, a factor of 6. The 19.1% headline is SWE-agent’s number and must never be quoted as “coding agents waste 19%”.

A scope correction emerged from the same work: 16.7% of SWE-agent edit steps sit in runs that produced **no patch at all**. Restricted to runs with a non-empty patch — the strictly comparable scope — the reference becomes **kept** 0.189 / **revised** 0.624 / **dead_end** 0.187 (196,651 edits, 22,194 runs). Both scopes are correct for their purpose; only the second is comparable across scaffolds, and every figure in this paper is labelled with which it uses.

7.2 Per-step test outcomes are too sparse to test the central claim

Per-step test results would be the instrument that could falsify our central claim. We built the parser and validated it hard: of 730 observations that provably contain test output, **730/730** recovered a verdict, **zero misses, zero false positives**. It still cannot answer the question, because **the data is too sparse** — only 0.55% of steps and 4.4% of runs contain a test outcome. It does find a real signal (runs that saw a test outcome passed the evaluator 3.0% of the time against 0.72%, a $\sim 4\times$ effect) but at 4.4% coverage it cannot falsify anything. Reported as a negative with a signal.

8 Retraction ledger

Eight claims failed tests written to falsify them. The three headline ones:

claim	what happened
“Failed runs localise better ”	Retracted. The metric is self-referential and inverts the sign. Corrected: solved 0.477 vs failed 0.407 on gold.
“0 false alarms at every budget”	Retracted as circular. Label and detector were one statistic; the numbers were the oracle’s. Gap reopened.
“Step index beats every learned monitor”	Retracted. The baseline <i>was</i> the run’s length — hindsight, unavailable online. Length-matched, the learned model wins.
“Waste is essentially unpredictable”	Weakened. True for a single monitor (0.539); a fitted model reaches 0.590 ± 0.011 , positive in all 12 learner/seed/form cells.
“84.3% of edits are wasted”	Corrected to 19.1%. The larger figure counted revision, which is work.
“The waste is not a phase”	Survived, but each required controls the naive comparison failed.

We also record that the **qualitative thesis is prior art** (§2). Our contribution is the measurement defect and the bounded-lever quantification, not the phenomenon.

9 Limitations

- **Small n in the causal experiment** (16–17 per arm). The robust results are the pre-registered inequality (P1) and the conditional success rates (P3), not pairwise p -values.

- **The task suite is single-defect mutations in eight real packages**, not SWE-bench instances: a controlled instrument, not a benchmark.
- **The mechanical instrument requires a comparable editor footer**; cross-framework generality is **not** claimed.
- **The inversion’s mechanism is unexplained** — three candidates tested, all failed.
- **Gold matching is basename-based**, and 4.10% of edit steps carry no filename, biasing the LOST share upward.
- **The prefix cutoff uses hindsight** ($L = \text{round}(f \cdot n_{\text{steps}})$), so the fully-online claim rests on the length-matched control.
- **Per-step test outcomes are too sparse (4.4% of runs)** to test the central claim from within this data.
- **Temperature-0 inference is not deterministic**: published measurement finds $\sim 9\%$ of per-instance outcomes flip across repeats, so the within-instance split is reported with runs-per-instance (mean 21.9).

10 Reproducibility

Every quantitative claim here is emitted by a script from a frozen artifact; none is typed by hand. Five automated gates run on every change:

gate	result
invariant tests	11/11
cross-artifact consistency + staleness	passes
headline claims vs the artifacts that produced them	28/28
one-page summary vs artifacts	0 mismatches
every cited artifact exists	all present

Corpora: SWE-agent trajectories (12 shards, 80,036 trajectories) with gold patches for 927 instances; Terminal-Bench trajectories (2 shards, 52,104 trials). Models: DeepSeek V4.1 Flash via the OpenCode Go gateway, temperature 0, total live-experiment spend **\$0.40**, every call logged.

11 Use of AI tools

A large language model (DeepSeek V4.1 Flash) was used as a **research assistant** throughout: writing analysis code, running statistical tests, fetching and verifying literature, and drafting this manuscript. It also served as the *subject* of the causal experiment in §5.

It was **not** used to generate data. All measurements come from public trajectory corpora and from real test suites executed locally. **Eight of its own claims were retracted when falsification tests succeeded**, and every retraction, bug and reversal is recorded in the project’s assistance log with the evidence that caused it; the ledger in §8 is a direct product of that record. The authors reviewed all code, verified all citations against primary sources, and take responsibility for the final claims.

References

- [1] Mehta et al. *Coherence Collapse: Diagnosing Why Code Agents Fail After Reaching the Right Code*. arXiv:2603.24631, 2026.
- [2] Majgaonkar et al. *Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories*. arXiv:2511.00197, ICSE 2026.
- [3] *FailForge: Distilling Procedural Competence from Persistent Failures into Code Agents*. arXiv:2608.08570, 2026.
- [4] *TraceProbe*. arXiv:2607.06184, 2026.

- [5] Mehta. *Confident and Wrong: Silent Semantic Failures in Coding Agents*. arXiv:2603.25764, 2026.
- [6] *TRIM: Reducing AI-Generated CodeSlop via Agent Trajectory Minimization*. arXiv:2607.18161, 2026.
- [7] Pham et al. *Terminating Local AI Agents Early to Save Energy in Consumer Devices*. arXiv:2605.15206.
- [8] Zhu et al. *Which Agent Causes Task Failures and When?* arXiv:2505.00212, ICML 2025.
- [9] *Failure as a Process*. arXiv:2607.09510, 2026.
- [10] *RedundancyBench*. arXiv:2605.29893, 2026.
- [11] *Temperature-0 nondeterminism on SWE-bench Verified*. arXiv:2607.09691, 2026.
- [12] *Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure*. arXiv:2604.02547, 2026.